# ClaudeSpy - iOS App

A modern iOS application using a **workspace + SPM package** architecture for clean separation between app shell and feature code.

## AI Assistant Rules Files

This template includes **opinionated rules files** for popular AI coding assistants. These files establish coding standards, architectural patterns, and best practices for modern iOS development using the latest APIs and Swift features.

### Included Rules Files

- **Claude Code**: `CLAUDE.md` - Claude Code rules
- **Cursor**: `.cursor/*.mdc` - Cursor-specific rules
- **GitHub Copilot**: `.github/copilot-instructions.md` - GitHub Copilot rules

### Customization Options

These rules files are **starting points** - feel free to:
- ✅ **Edit them** to match your team's coding standards
- ✅ **Delete them** if you prefer different approaches
- ✅ **Add your own** rules for other AI tools
- ✅ **Update them** as new iOS APIs become available

### What Makes These Rules Opinionated

- **No ViewModels**: Embraces pure SwiftUI state management patterns
- **Swift 6+ Concurrency**: Enforces modern async/await over legacy patterns
- **Latest APIs**: Recommends iOS 18+ features with optional iOS 26 guidelines
- **Testing First**: Promotes Swift Testing framework over XCTest
- **Performance Focus**: Emphasizes @Observable over @Published for better performance

**Note for AI assistants**: You MUST read the relevant rules files before making changes to ensure consistency with project standards.

## Project Architecture

```
ClaudeSpy/
├── ClaudeSpy.xcworkspace/              # Open this file in Xcode
├── ClaudeSpy.xcodeproj/                # App shell project
├── ClaudeSpy/                          # App target (minimal)
│   ├── Assets.xcassets/                # App-level assets (icons, colors)
│   ├── ClaudeSpyApp.swift              # App entry point
│   └── ClaudeSpy.xctestplan            # Test configuration
├── ClaudeSpyPackage/                   # 🚀 Primary development area
│   ├── Package.swift                   # Package configuration
│   ├── Sources/ClaudeSpyFeature/       # Your feature code
│   └── Tests/ClaudeSpyFeatureTests/    # Unit tests
└── ClaudeSpyUITests/                   # UI automation tests
```

## Key Architecture Points

### Workspace + SPM Structure

- **App Shell**: `ClaudeSpy/` contains minimal app lifecycle code
- **Feature Code**: `ClaudeSpyPackage/Sources/ClaudeSpyFeature/` is where most development happens
- **Separation**: Business logic lives in the SPM package, app target just imports and displays it

### Buildable Folders (Xcode 16)

- Files added to the filesystem automatically appear in Xcode
- No need to manually add files to project targets
- Reduces project file conflicts in teams

## Development Notes

### Code Organization

Most development happens in `ClaudeSpyPackage/Sources/ClaudeSpyFeature/` - organize your code as you prefer.

### Public API Requirements

Types exposed to the app target need `public` access:

```
public struct NewView: View {
    public init() {}

    public var body: some View {
        // Your view code
    }
}
```

### Adding Dependencies

Edit `ClaudeSpyPackage/Package.swift` to add SPM dependencies:

```
dependencies: [
    .package(url: "https://github.com/example/SomePackage", from: "1.0.0")
],
targets: [
    .target(
        name: "ClaudeSpyFeature",
        dependencies: ["SomePackage"]
    ),
]
```

### Test Structure

- **Unit Tests**: `ClaudeSpyPackage/Tests/ClaudeSpyFeatureTests/` (Swift Testing framework)
- **UI Tests**: `ClaudeSpyUITests/` (XCUITest framework)
- **Test Plan**: `ClaudeSpy.xctestplan` coordinates all tests

## Configuration

### XCConfig Build Settings

Build settings are managed through **XCConfig files** in `Config/`:
- `Config/Shared.xcconfig` - Common settings (bundle ID, versions, deployment target)
- `Config/Debug.xcconfig` - Debug-specific settings
- `Config/Release.xcconfig` - Release-specific settings
- `Config/Tests.xcconfig` - Test-specific settings

### Entitlements Management

App capabilities are managed through a **declarative entitlements file**:
- `Config/ClaudeSpy.entitlements` - All app entitlements and capabilities
- AI agents can safely edit this XML file to add HealthKit, CloudKit, Push Notifications, etc.
- No need to modify complex Xcode project files

## Asset Management

- **App-Level Assets**: `ClaudeSpy/Assets.xcassets/` (app icon, accent color)
- **Feature Assets**: Add `Resources/` folder to SPM package if needed

## SPM Package Resources

To include assets in your feature package:

```
.target(
    name: "ClaudeSpyFeature",
    dependencies: [],
    resources: [.process("Resources")]
)
```

## Generated with XcodeBuildMCP

This project was scaffolded using XcodeBuildMCP, which provides tools for AI-assisted iOS development workflows.